Student guide | Lesson 2 of 5

Keep moving and hit the wall

The square moves on its own and resets at the edge.

Name: _____ Date: _____

Today you will make

The square now moves by itself. When it leaves the screen, the game restarts it in the middle. This is called a **game loop**.

Words you will meet

Word	What it means
state	Information the program remembers, like which way we are going.
condition	A question that is true or false. We use it with **if**.
game loop	A function that runs again and again to update the game.
abs()	Gives the distance from zero. abs(-5) is 5.
millisecond	One thousandth of a second. 100 ms is a tenth of a second.

The big idea

In Lesson 1 the keys moved the square. Now the keys only **change a variable** called direction. The game loop reads it and does the moving. Keys -> direction -> loop moves the head.

Before you start

- Open snake.py from Lesson 1 and choose **File > Save As**. Save it as **snake2.py**.
- Keep the old file so you can go back if you break something.

Step 1: Add a limit and turn off auto-drawing

Add LIMIT under STEP. Add tracer under screen.setup.

```
STEP = 20
LIMIT = 280       # how far from the centre counts as "out"

screen.setup(600, 600)
screen.tracer(0)  # we will redraw the screen ourselves
```

What it does

- **LIMIT = 280** is the edge of the play area.
- **tracer(0)** stops turtle drawing every tiny change. Later we call **screen.update()** once per frame so the game looks smooth.

Try it: Do not run yet. With tracer(0) nothing appears until Step 5.

Step 2: Give the head a memory

Add one line directly under head.penup():

```
head.direction = "stop"   # the head remembers which way it is going
```

What it does

- **direction** is a value stored on the head. It starts as "stop".

Step 3: Change the four key functions

Delete your old go_up, go_down, go_left and go_right. Replace them with these:

```
def go_up():    head.direction = "up"
def go_down():  head.direction = "down"
def go_left():  head.direction = "left"
def go_right(): head.direction = "right"
```

What it does

- They no longer move the square. They only remember which way it should go.

Step 4: Write move_head

Add this under the key functions:

```
def move_head():
    if head.direction == "up":    head.sety(head.ycor() + STEP)
    if head.direction == "down":  head.sety(head.ycor() - STEP)
    if head.direction == "left":  head.setx(head.xcor() - STEP)
    if head.direction == "right": head.setx(head.xcor() + STEP)
```

What it does

- It checks the direction and moves one STEP that way.
- Each **if** asks a true-or-false question. Only the matching one runs.

Step 5: Write the game loop

Add this under move_head. Then add **game_loop()** on its own line **just above screen.mainloop()**.

```
def game_loop():
    if abs(head.xcor()) > LIMIT or abs(head.ycor()) > LIMIT:
        head.goto(0, 0)
        head.direction = "stop"

    move_head()
    screen.update()
    screen.ontimer(game_loop, 100)   # run again in 100 ms
```

What it does

- The first **if** checks if the head is too far from the centre. **abs()** means -300 and 300 both count as far away.
- **goto(0, 0)** puts the head back in the middle.
- **screen.update()** draws the new frame.
- **ontimer(game_loop, 100)** runs game_loop again after 100 ms. That is 10 frames per second.

Try it: Run it. Press an arrow key. The square should glide until it leaves the screen, then jump back to the middle and stop.

Think about it: Why does the square not move when the program starts?

Your answer: _____

Think about it: What happens if you change 100 to 50? What about 200?

Your answer: _____

Challenges (pick one, or do them all)

[ ] Change 100 to 50, then to 200. Which feels hardest to play?

[ ] Change LIMIT to 200. What changes?

[ ] Make the head turn red when it resets. Hint: head.color("red").

Stuck? Check here first

What you see	Likely cause and fix
The square does not move at first	That is correct. Direction starts as "stop". Press an arrow key.
Nothing appears or the window looks frozen	screen.update() is missing from game_loop.
The square moves once and stops	screen.ontimer(...) is missing, or game_loop() is not called before mainloop().
NameError: name 'LIMIT' is not defined	You skipped Step 1.

I can...

[ ] store information that my program remembers.

[ ] explain why keys change direction instead of moving the head.

[ ] use if to check a condition.

[ ] make a function run again and again with ontimer.

Check your work: the complete code

Compare your file with this one, line by line. If yours does not run, the mistake is usually a typo, a missing colon, or indentation that is not lined up.

```
import turtle

STEP = 20
LIMIT = 280

screen = turtle.Screen()
screen.title("Snake")
screen.bgcolor("black")
screen.setup(600, 600)
screen.tracer(0)          # we'll redraw manually

head = turtle.Turtle("square")
head.color("lime")
head.penup()
head.direction = "stop"   # the head remembers which way it's going

def go_up():    head.direction = "up"
def go_down():  head.direction = "down"
def go_left():  head.direction = "left"
def go_right(): head.direction = "right"

def move_head():
    if head.direction == "up":    head.sety(head.ycor() + STEP)
    if head.direction == "down":  head.sety(head.ycor() - STEP)
    if head.direction == "left":  head.setx(head.xcor() - STEP)
    if head.direction == "right": head.setx(head.xcor() + STEP)

def game_loop():
    if abs(head.xcor()) > LIMIT or abs(head.ycor()) > LIMIT:
        head.goto(0, 0)
        head.direction = "stop"

    move_head()
    screen.update()
    screen.ontimer(game_loop, 100)   # run again in 100 ms

screen.listen()
screen.onkeypress(go_up, "Up")
screen.onkeypress(go_down, "Down")
screen.onkeypress(go_left, "Left")
screen.onkeypress(go_right, "Right")

game_loop()
screen.mainloop()
```